smooth - Experimental Data Smoothing

Version 5.11.41 | June 11, 2026

A command-line tool for smoothing noisy experimental data and computing derivatives. Implements four methods: polynomial fitting, Savitzky-Golay filtering, Tikhonov regularization, and Butterworth low-pass filtering. Reads two-column ASCII data, outputs smoothed results. Works as a Unix filter.

Contents

Practical Guide

Technical Appendices

Quick Start

```
make                                    # Compile
./smooth -g data.txt                    # Analyze your data grid
./smooth -m 2 -l auto data.txt         # Smooth with Tikhonov (auto parameter)
```

Installation

Requirements

- C compiler (gcc, clang)
- LAPACK and BLAS libraries
- Make (optional, but recommended)

Compilation using Make

```
make                # Standard compilation (clang, -O2)
make debug          # Debug build (-g -O0)
```

```
make test             # Build and run 116 unit tests
make test-valgrind    # Run tests with memory leak detection
make clean            # Clean build artifacts
make install-user     # Install to ~/bin
make install          # Install system-wide (requires root)
make help             # Show all available targets
```

Manual Compilation

```
gcc -o smooth smooth.c polyfit.c savgol.c tikhonov.c butterworth.c \
    grid_analysis.c timestamp.c parser.c -llapack -lblas -lm -O2
```

Usage Reference

```
./smooth [options] [data_file|-]
```

Input Options: - `data_file` - read data from file - `-` - read data from stdin (standard input) - omit argument - read data from stdin (default when no file specified)

Options

Option	Description
`-m {0|1|2|3}`	Method: polyfit | savgol | tikhonov | butterworth
`-n N`	Smoothing window size (polyfit, savgol)
`-p P`	Polynomial degree (polyfit, savgol, max 12)
`-l λ`	Regularization parameter (tikhonov), use `-l auto` for GCV
`-f fc`	Normalized cutoff frequency (butterworth, 0 < fc < 1.0). Default: `auto` (Morozov's discrepancy principle); pass a numeric `fc` to override
`-T`	Timestamp mode: a column holds an RFC3339 timestamp (default position 1, y at position 2; both adjustable via `-k`)
`-k M` or `-k N:M`	Column selection: `M` sets the y-data column (x defaults to column 1); `N:M` sets x at column N and y at column M. Default: `1:2`. Columns are 1-indexed; N and M must differ. In `-T` mode, N is the **timestamp** column (still column 1 by default), and the timestamp counts as a single logical column even when its space-separated form spans two whitespace tokens.
`-d`	Include first derivative in output
`-g`	Show detailed grid uniformity analysis
`-h, -?`	Show help (options, methods, examples) and exit

Notes: - Polynomial degrees > 6 may generate numerical stability warnings. - First derivative output is optional. Without `-d`, only smoothed values are output. - In timestamp mode (`-T`), timestamps are converted to relative time in seconds for smoothing but preserved in output. Derivatives are dy/dt where t is in seconds.

Input Format

ASCII data with one record per line. By default, column 1 is x and column 2 is y; extra columns are ignored. Use `-k M` to pick a different y column, or `-k N:M` to pick both x and y columns (e.g. `-k 1:4` uses column 1 as x and column 4 as y). Comments (lines starting with #) are stripped automatically. Data must have strictly monotonic increasing x-values. If a line has fewer columns than requested, the program exits with an error identifying the offending line.

Columns are split on whitespace; each whitespace-separated token is one logical column. Tokens that are not fully numeric (e.g. an ISO 8601 timestamp `2026-04-29T11:40:00`, a label, or a partially numeric `1.5e2x`) hold the column position but carry no value. If the selected x or y column lands on such a token, or on `NaN`/`Inf`, the row is skipped and a `# Skipped N data row(s) ...` summary is written to stdout. Other columns may be non-numeric without affecting parsing.

In timestamp mode (`-T`), the same `-k N:M` selection applies, but column 1 (the default `N`) is the timestamp instead of a numeric x-value. The timestamp is treated as a single logical column even though the space-separated form (`YYYY-MM-DD HH:MM:SS.fff`) spans two whitespace tokens; the T-separated form (`YYYY-MM-DDTHH:MM:SS.fff`) is one token. Example: with input rows `ID 2025-09-25 14:06:06.390 25.5 100.2 980.1`, `-T -k 2:5` selects the timestamp at logical column 2 and y at logical column 5 (= `100.2`). The same skip-and-summarize behavior applies if the y-token is non-numeric or `NaN`/`Inf`; rows with an unparsable timestamp are reported separately by the timestamp parser.

Unix Filter Usage

The program works as a standard Unix filter in pipe chains:

```
cat data.txt | smooth -m 1 -n 5 -p 2       # Pipe input
smooth -m 2 -l 0.01 < input.txt > out.txt   # Redirection
command | smooth -m 3 -f 0.15 | gnuplot     # Pipeline
```

Method Selection Guide

Decision Tree

```
Not sure which method to use?
+-- Use TIKHONOV with -l auto (safest, most automatic)
    ./smooth -m 2 -l auto -d data.txt

Is your grid uniform (CV < 0.05)?
|-- YES: Multiple good options:
```

```
|   |-- SAVGOL: Best for polynomial signals with derivatives
|   |   ./smooth -m 1 -n 9 -p 3 -d data.txt
|   |-- BUTTERWORTH: Best for frequency-domain interpretation
|   |   ./smooth -m 3 -f 0.15 data.txt
|   +-- TIKHONOV: Universal choice with auto parameters
|       ./smooth -m 2 -l auto -d data.txt
|
+-- NO (non-uniform grid):
    +-- TIKHONOV handles this correctly and automatically
        ./smooth -m 2 -l auto -d data.txt

Need local adaptability / variable curvature?
+-- Use POLYFIT regardless of grid
    ./smooth -m 0 -n 7 -p 2 -d data.txt

Need frequency-domain control?
+-- Use BUTTERWORTH (requires uniform grid)
    ./smooth -m 3 -f 0.15 data.txt
```

Method Summaries

Polynomial Fitting (POLYFIT, `-m 0`) - Fits a polynomial in a sliding window around each data point using SVD least squares. Adapts well to variable curvature and works with non-uniform grids. Provides high-quality derivatives. Best when you need local adaptability.

Savitzky-Golay (SAVGOL, `-m 1`) - Pre-computes universal convolution coefficients for optimal polynomial smoothing. Very fast for large datasets. Requires uniform grid spacing (automatically checked). Best for polynomial signals on uniform grids when you need derivatives and computational efficiency.

Tikhonov Regularization (TIKHONOV, `-m 2`) - Global optimization that balances data fidelity against smoothness. Automatic parameter selection via GCV. Handles uniform and non-uniform grids correctly. Best as a general-purpose method when you want automatic, robust smoothing.

Butterworth Filter (BUTTERWORTH, `-m 3`) - Digital low-pass filter that removes high-frequency noise. Zero phase distortion via forward-backward filtering. Requires uniform grid. Provides first derivatives (`-d`) via 5-point stencils on the filtfilt output. Best when you want frequency-domain control of smoothing.

Comparison Tables

Smoothing Quality

Property	POLYFIT	SAVGOL	TIKHONOV	BUTTERWORTH
Local adaptability	*****	****	**	**
Extreme preservation	****	*****	***	***
Noise robustness	***	****	*****	*****

Property	POLYFIT	SAVGOL	TIKHONOV	BUTTERWORTH
Derivative quality	*****	*****	***	***
Boundary behavior	**	***	****	***
Non-uniform grids	***	[X]	*****	**
Ease of use	****	****	*****	****
Parameter selection	Manual	Manual	Auto (GCV)	Manual
Frequency control	No	No	No	Yes
Phase distortion	N/A	N/A	N/A	Zero

Key: [X] = Not suitable (automatically rejected)

Grid Type Compatibility

Grid Type	POLYFIT	SAVGOL	TIKHONOV	BUTTERWORTH
Uniform (CV <= 0.01)	[OK]	[OK]	[OK]	[OK]
Nearly uniform (CV < 0.05)	[OK]	[WARNING]	[OK]	[OK]
Moderately non-uniform (0.05 <= CV < 0.15)	[OK]	[X]	[OK]	[WARNING]
Non-uniform (CV >= 0.15)	[WARNING]	[X]	[OK]	[X]

Legend: [OK] = Recommended, [WARNING] = Usable with caution, [X] = Rejected or not recommended

Computational Complexity

Method	Time	Memory	Scalability
POLYFIT	$O(n \cdot p^3)$	$O(p^2)$	Good for small p
SAVGOL	$O(p^3) + O(n \cdot w)$	$O(w)$	Excellent for large n
TIKHONOV	$O(n)$	$O(n)$	Excellent
BUTTERWORTH	$O(n)$	$O(n)$	Excellent

Note: w = window size, p = polynomial degree (<=12), n = number of data points.

Parameter Tuning

Window Size (n) for POLYFIT/SAVGOL

```
n = 2*k + 1    (odd number)
```

```
Recommendations:
- Low noise: n = 5-9
- Medium noise: n = 9-15
- High noise: n = 15-25

Rule of thumb: n >= 2p + 3
```

Polynomial Degree (p) for POLYFIT/SAVGOL

```
- Linear trends: p = 1-2
- Smooth curves: p = 2-3
- Complex signals: p = 3-4
- Advanced applications: p = 5-8
- Maximum: p <= 12
- Recommended maximum: p < n/2

Note: Degrees > 6 may cause numerical instability warnings.
```

Lambda for TIKHONOV

Automatic selection (recommended):

```
./smooth -m 2 -l auto data.txt
```

Uses Generalized Cross Validation (GCV) to find optimal lambda.

Manual selection:

Data Characteristics	Recommended lambda	Reasoning
Low noise, important details	0.001 - 0.01	Preserve features
Moderate noise	0.01 - 0.1	Balanced (default: 0.1)
High noise	0.1 - 1.0	Strong smoothing
Very noisy, global trends	1.0 - 10.0	Maximum smoothing

Iterative refinement:

```
# Start with automatic
./smooth -m 2 -l auto data.txt

# If result is over-smoothed (details lost):
./smooth -m 2 -l 0.01 data.txt

# If result is under-smoothed (still noisy):
./smooth -m 2 -l 1.0 data.txt
```

Diagnostic output — check functional balance in output comments:

```
# Functional J = 1.234e+02 (Data: 5.67e+01 + Regularization: 6.67e+01)
# Data/Total ratio = 0.460, Regularization/Total ratio = 0.540
```

Good balance: both terms contribute 30-70% of total. Data term > 95% means under-smoothed. Regularization term > 95% means over-smoothed.

Grid-dependent considerations: For highly non-uniform grids (CV > 0.2), start with more conservative (larger) lambda. GCV may be less accurate — check results visually.

Cutoff Frequency (fc) for BUTTERWORTH

fc Value	Smoothing Strength	When to Use
0.01 - 0.05	**Very strong**	Extremely noisy data, only global trends matter
0.05 - 0.15	**Moderate**	Typical experimental data with noise
0.15 - 0.30	**Light**	Good quality data, preserve features
> 0.30	**Minimal**	Low noise, want to keep almost everything

Quick start: The default is `-f auto` (described below). To override, pass a numeric value: start with fc = 0.15. Too noisy after smoothing? Decrease fc. Lost important details? Increase fc.

Automatic selection (default): Without `-f` (or with explicit `-f auto`) the program selects fc via **Morozov's discrepancy principle**. It estimates noise $\hat{\sigma}$ from the MAD of second differences, then scans candidate cutoffs $\{0.02, 0.05, 0.1, 0.2, 0.35, 0.5\}$ in increasing order and picks the smallest fc whose residual std does not exceed a tolerance multiple of $\hat{\sigma}$ (i.e. the most aggressive smoothing that still leaves only noise-sized residuals). If no candidate satisfies the criterion (very high SNR or pathological data), falls back to fc = 0.2 with a `# Auto cutoff: ... fallback ...` notice. The selected fc is reported in the output header. For unusual data, manual tuning is still recommended.

Physical interpretation:

fc = f_cutoff / f_Nyquist, where f_Nyquist = f_sample / 2 and f_sample = 1 / h_avg.

Example: Data with spacing h_avg = 0.1 sec → f_sample = 10 Hz, f_Nyquist = 5 Hz. If fc = 0.2, then f_cutoff = 0.2 x 5 = 1 Hz (removes frequencies above ~1 Hz).

Usage Examples

Basic Syntax

```
# Read from file
./smooth -m 0 -n 7 -p 2 data.txt
```

```
# Read from stdin (pipe)
cat data.txt | ./smooth -m 0 -n 7 -p 2

# Read from stdin (explicit)
./smooth -m 0 -n 7 -p 2 -

# Read from stdin (redirection)
./smooth -m 0 -n 7 -p 2 < data.txt
```

Method Examples

```
# Polynomial fitting (smoothed values only)
./smooth -m 0 -n 7 -p 2 data.txt

# Polynomial fitting with derivatives
./smooth -m 0 -n 7 -p 2 -d data.txt

# Savitzky-Golay (smoothed values only)
# NOTE: Will be rejected if grid is non-uniform!
./smooth -m 1 -n 9 -p 3 data.txt

# Savitzky-Golay with derivatives
./smooth -m 1 -n 9 -p 3 -d data.txt

# Tikhonov with automatic lambda (RECOMMENDED)
./smooth -m 2 -l auto data.txt

# Tikhonov with automatic lambda and derivatives
./smooth -m 2 -l auto -d data.txt

# Tikhonov with manual lambda
./smooth -m 2 -l 0.01 data.txt

# Tikhonov with manual lambda and derivatives
./smooth -m 2 -l 0.01 -d data.txt

# Butterworth with manual cutoff frequency
./smooth -m 3 -f 0.15 data.txt

# Butterworth with automatic cutoff (Morozov's discrepancy principle)
./smooth -m 3 -f auto data.txt

# Grid analysis only (exits after analysis)
./smooth -g data.txt
```

Timestamp Mode Examples

```
# Tikhonov smoothing with automatic lambda selection
./smooth -T -m 2 -l auto timeseries.dat

# Tikhonov with derivatives (dy/dt in seconds)
./smooth -T -m 2 -l 0.01 -d timeseries.dat

# Savitzky-Golay filter (requires nearly uniform time spacing)
./smooth -T -m 1 -n 5 -p 2 sensor_data.txt

# Polyfit with derivatives
./smooth -T -m 0 -n 7 -p 3 -d measurements.csv

# Butterworth filter (add -d for derivatives via 5-point stencils)
./smooth -T -m 3 -f 0.15 signal.dat
```

Input format for timestamp mode:

```
# Space separator format
2025-09-25 14:06:06.390  0.02128
2025-09-25 14:06:06.391  0.02110
2025-09-25 14:06:06.763  0.02230

# T separator format (RFC3339)
2025-09-25T14:06:06.390  0.02128
2025-09-25T14:06:06.391  0.02110
2025-09-25T14:06:06.763  0.02230

# Multi-column with -k: extra fields before/after timestamp
# (timestamp at logical column 2, y at logical column 5)
sensorA  2025-09-25 14:06:06.390  25.5  100.2  980.1
sensorA  2025-09-25 14:06:06.391  25.6  100.3  980.0
# invocation: ./smooth -T -k 2:5 -m 2 -l auto data.txt
```

Working with Non-uniform Grids

```
# First, analyze your grid
./smooth -g nonuniform_data.txt

# Tikhonov uses a single integral-measure discretization for all grids
# (uniform and non-uniform alike); there is no CV-based method switch.
# Apply smoothing with automatic parameter selection:
./smooth -m 2 -l auto nonuniform_data.txt

# For highly non-uniform grids (CV > 0.2), you may see:
# "WARNING: Highly non-uniform grid detected!"
# "GCV trace approximation may be less accurate."
# In this case, try manual lambda or check results visually.
```

Unix Filter Examples

```
# Simple pipe from cat
cat noisy_data.txt | ./smooth -m 1 -n 5 -p 2 > smoothed.txt

# Extract columns, smooth, and plot
awk '{print $1, $3}' experiment.dat | ./smooth -m 2 -l auto | gnuplot -p plot.gp

# Process multiple files
for f in data_*.txt; do
  cat "$f" | ./smooth -m 3 -f 0.15 > "smooth_$f"
done

# Filter out comments, smooth, extract columns
grep -v '^#' raw.txt | ./smooth -m 2 -l 0.01 | awk '{print $1, $2}' > final.txt

# Combine with other tools
./generate_data | ./smooth -m 1 -n 7 -p 3 | ./analyze_results

# Standard input/output redirection
./smooth -m 0 -n 5 -p 2 < input.dat > output.dat 2> errors.log
```

Typical Workflow

1. **Analyze your grid:**

   ```
   ./smooth -g data.txt
   ```

2. **Choose method based on grid:**

   ```
   # For uniform grids (CV < 0.05):
   ./smooth -m 1 -n 9 -p 3 -d data.txt

   # For non-uniform grids:
   ./smooth -m 2 -l auto -d data.txt
   ```

3. **Refine parameters if needed:**

   ```
   # If automatic lambda gives over-smoothing:
   ./smooth -m 2 -l 0.01 -d data.txt

   # If under-smoothing:
   ./smooth -m 2 -l 1.0 -d data.txt
   ```

4. **For publication graphics:**

   ```
   ./smooth -m 2 -l auto -d data.txt > publication_data.txt
   ```

Grid Analysis

The `-g` flag provides detailed grid uniformity statistics to help choose appropriate smoothing methods and parameters.

```
./smooth -g data.txt
```

Example Output

```
# Grid uniformity analysis:
#   n = 1000 points
#   h_min = 9.500000e-03, h_max = 1.200000e-02, h_avg = 1.000000e-02
#   CV = 0.052
#   Grid type: NON-UNIFORM
#   Uniformity score: 0.90
#   Standard deviation: 5.200000e-04
#   Detected clusters: 0
#   Recommendation: Grid is nearly uniform - standard methods work well
```

Uniformity Thresholds

CV Range	Grid Type	Effect on Methods
CV <= 0.01	Uniform	All methods work optimally
CV < 0.05	Nearly uniform	SAVGOL works with warning; BUTTERWORTH info note; all others fine
0.05 <= CV < 0.15	Moderately non-uniform	SAVGOL rejected; BUTTERWORTH warns; TIKHONOV uses integral-measure Gram matrix
0.15 <= CV < 0.20	Non-uniform	SAVGOL rejected; BUTTERWORTH rejected; TIKHONOV uses integral-measure Gram matrix
CV >= 0.20	Highly non-uniform	SAVGOL rejected; BUTTERWORTH rejected; TIKHONOV uses integral-measure Gram matrix (GCV trace less accurate)

The coefficient of variation (CV) is defined as: $CV = \sigma(h)/h_{\text{avg}}$

Output Format

Without `-d` flag:

```
# Data smooth - Tikhonov regularization with lambda = 1e-01
# Functional J = 1.234e+02 (Data: 5.67e+01 + Regularization: 6.67e+01)
# Data/Total ratio = 0.460, Regularization/Total ratio = 0.540
```

```
#    x          y
  0.00000E+00  1.00000E+00
  1.00000E+00  2.71828E+00
  ...
```

With `-d` flag:

```
# Data smooth - Tikhonov regularization with lambda = 1e-01
# Functional J = 1.234e+02 (Data: 5.67e+01 + Regularization: 6.67e+01)
# Data/Total ratio = 0.460, Regularization/Total ratio = 0.540
#    x          y          y'
  0.00000E+00  1.00000E+00  1.00000E+00
  1.00000E+00  2.71828E+00  2.71828E+00
  ...
```

Timestamp mode output (with `-T` flag):

```
# Data smooth - Tikhonov regularization with lambda = 1e-02
# Functional J = 1.07e-03 (Data: 8.33e-04 + Regularization: 2.39e-04)
# Data/Total ratio = 0.777, Regularization/Total ratio = 0.223
# Derivative units: dy/dt (t in seconds)
#    timestamp          y          y'
2025-09-25 14:06:06.390 0.000816394 -0.00204621
2025-09-25 14:06:06.391 0.000814348  0.0542818
2025-09-25 14:06:06.763  0.0210635  0.0284901
  ...
```

In timestamp mode, the original timestamp format from input is preserved exactly in output. Values use general format (`%10.6lG`) for numeric data.

Appendix A: Mathematical Foundations

A.1 Polynomial Fitting (POLYFIT)

General Smoothing Problem Experimental data often contains random noise:

$$y_{\text{obs}}(x_i) = y_{\text{true}}(x_i) + \varepsilon_i$$

The goal of smoothing is to estimate y_{true} while suppressing ε_i and preserving physically relevant signal properties.

Local Polynomial Fitting The POLYFIT method uses local polynomial fitting with least squares method in a sliding window.

Problem: For each point x_i, we fit a polynomial of degree p to the surrounding n points:

$$P(x) = a_0 + a_1(x - x_i) + a_2(x - x_i)^2 + \cdots + a_p(x - x_i)^p$$

Optimization criterion:

$$\min \sum_{j \in [i-n/2,\, i+n/2]} \left[y_j - P(x_j) \right]^2$$

Least Squares Solution via SVD The polynomial coefficients are found by solving an **overdetermined linear system** using **Singular Value Decomposition (SVD)**:

$$V \cdot \mathbf{a} = \mathbf{y}_{\text{window}}$$

where V is the Vandermonde matrix with $V_{j,k} = (x_j - x_i)^k$:

$$V = \begin{pmatrix} 1 & (x_0 - x_i) & (x_0 - x_i)^2 & \cdots & (x_0 - x_i)^p \\ 1 & (x_1 - x_i) & (x_1 - x_i)^2 & \cdots & (x_1 - x_i)^p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & (x_n - x_i) & (x_n - x_i)^2 & \cdots & (x_n - x_i)^p \end{pmatrix}$$

$\mathbf{a} = [a_0, a_1, \ldots, a_p]^T$ (polynomial coefficients), $\mathbf{y}_{\text{window}} = [y_{i-n/2}, \ldots, y_{i+n/2}]^T$

Why SVD instead of Normal Equations?

The implementation uses LAPACK's `dgelss` (SVD decomposition) rather than forming normal equations $(V^T V)\mathbf{a} = V^T \mathbf{y}$:

1. **Numerical stability:** SVD avoids squaring the condition number ($\kappa(V^T V) = \kappa(V)^2$)
2. **Automatic regularization:** Singular values below rcond $\cdot\, \sigma_{\text{max}}$ are truncated
3. **Rank detection:** Provides effective rank for diagnosing ill-conditioning
4. **Robustness:** Handles high polynomial degrees (p > 6) more reliably

SVD truncation parameter: rcond $= 10^{-10}$ - Singular values $\sigma_i < 10^{-10} \cdot \sigma_{\text{max}}$ are treated as zero - Provides implicit Tikhonov-style regularization - Conservative threshold ensures stability without over-regularization

Derivative Computation Derivatives are computed analytically from polynomial coefficients:

$$f(x_i) = a_0, \qquad f'(x_i) = a_1, \qquad f''(x_i) = 2a_2$$

Edge Handling At edges, asymmetric windows are used with extrapolation of the fitted polynomial:

$$f(x_k) = \sum_{m=0}^{p} a_m \cdot (x_k - x_{n/2})^m$$

Characteristics **Advantages:** - Excellent local approximation - Analytical computation of derivatives of any order - Adaptable to changes in curvature - Good preservation of local extrema - Works with moderately non-uniform grids - **Numerically stable:** SVD decomposition handles ill-conditioned systems - **Automatic regularization:** Implicit truncation of small singular values - **Diagnostic feedback:** Reports condition number and effective rank

Disadvantages: - Sensitive to outliers - Boundary effects at edges - Possible Runge oscillations for high polynomial degrees (p > 6) - Numerical instability warnings for degrees > 6 (but handled gracefully by SVD) - Computationally expensive: O(n·p³) due to per-point SVD

A.2 Savitzky-Golay Filter (SAVGOL)

Theoretical Foundations The Savitzky-Golay filter is an optimal linear filter for smoothing and derivatives based on local polynomial regression. The key innovation is pre-computation of convolution coefficients.

Fundamental principle: For given parameters (window size, polynomial degree, derivative order), there exist universal coefficients c_k such that:

$$f^{(d)}(x_i) = \sum_{k=-n_L}^{n_R} c_k \cdot y_{i+k}$$

Key Difference from POLYFIT While both SAVGOL and POLYFIT use polynomial approximation, they differ fundamentally:

POLYFIT approach: - For each data point, fits a new polynomial to the surrounding window - Solves the least squares problem individually for each point - Coefficients of the polynomial change with each window position - Computationally intensive: O(n·p³)

SAVGOL approach (Method of Undetermined Coefficients): - Recognizes that for equidistant grids, the filter coefficients are translation-invariant - Uses the **method of undetermined coefficients** to pre-compute universal weights - These weights depend only on the window geometry, not on the actual data values - Applies the same weights as a linear convolution across all data points - Computationally efficient: O(p³) once, then O(n·w) for application

Grid Uniformity Requirement The mathematical foundation of SG filter assumes **uniformly spaced data points**. The method is based on fitting polynomials in normalized coordinate space where points are at integer positions: {..., -2, -1, 0, 1, 2, ...}.

Uniformity Check:

$$CV = \frac{\sigma(h)}{h_{\text{avg}}}$$

CV range	Decision
$CV > 0.05$	REJECT — Grid too non-uniform for SG
$CV > 0.01$	WARNING — Nearly uniform, proceed with caution
$CV \leq 0.01$	OK — Grid sufficiently uniform

What happens when grid is rejected:

```
======================================
ERROR: Savitzky-Golay method not suitable for non-uniform grid!
======================================
Grid analysis:
  Coefficient of variation (CV) = 0.2341
  Threshold for uniformity = 0.0500

RECOMMENDED ALTERNATIVES:
  1. Use Tikhonov method: -m 2 -l auto
     (Works correctly with non-uniform grids)
  2. Use Polyfit method: -m 0 -n 5 -p 2
     (Local fitting, less sensitive to spacing)
  3. Resample your data to uniform grid before smoothing
```

The Method of Undetermined Coefficients The Savitzky-Golay method seeks a linear combination of data points:

$$\hat{y}_0 = c_{-n_L} \cdot y_{-n_L} + \cdots + c_0 \cdot y_0 + \cdots + c_{n_R} \cdot y_{n_R}$$

where the coefficients c_k are "undetermined" and must satisfy the condition that the filter exactly reproduces polynomials up to degree p.

The key insight: For a given window configuration and polynomial degree, these coefficients can be determined once and applied universally - but only on uniform grids!

Coefficient Derivation Coefficients are derived from the condition that the filter must exactly reproduce polynomials up to degree p.

Moment conditions:

$$\sum_{j=-n_L}^{n_R} c_j \cdot j^m = \delta_{m,d} \cdot d! \qquad \text{for } m = 0, 1, \ldots, p$$

where $\delta_{m,d}$ is the Kronecker delta, d is the derivative order, and $d!$ is factorial.

This leads to a system of linear equations where the unknowns are the filter coefficients c_j.

Matrix Formulation The coefficients are found by solving a **normal equations system** (not a Vandermonde system):

$$A \cdot \beta = \mathbf{b}$$

where A is a symmetric $(p+1) \times (p+1)$ moment matrix with $A_{i,j} = \sum_{k=-n_L}^{n_R} k^{i+j}$:

$$A = \begin{pmatrix} \sum k^0 & \sum k^1 & \sum k^2 & \cdots & \sum k^p \\ \sum k^1 & \sum k^2 & \sum k^3 & \cdots & \sum k^{p+1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \sum k^p & \sum k^{p+1} & \cdots & \cdots & \sum k^{2p} \end{pmatrix}$$

and the right-hand side vector: $b_j = \delta_{j,d} \cdot d!$

This results in a symmetric positive definite $(p+1) \times (p+1)$ matrix. The filter coefficients are then:

$$c_k = \sum_{j=0}^{p} \beta_j \cdot k^j \qquad \text{for } k = -n_L, \ldots, n_R$$

Note: This formulation through normal equations is mathematically equivalent to least-squares polynomial fitting but more efficient computationally.

Computational Efficiency Example for 10,000 data points, window size 21, polynomial degree 4: - **POLYFIT:** Must solve 10,000 separate 5x5 linear systems - **SAVGOL:** - Solves ONE 5x5 system for central points (pre-computed coefficients) - Performs 9,980 simple weighted sums (fast convolution) - Solves 20 boundary systems (asymmetric windows at edges) - **Net result:** ~500x faster for large datasets

Derivative Scaling The derivative coefficients computed by `savgol_coefficients()` assume **unit spacing** (normalized integer coordinates). For physical derivatives on real grids, the results must be scaled:

$$\left.\frac{dy}{dx}\right|_{\text{physical}} = \frac{1}{h_{\text{avg}}} \cdot \left.\frac{dy}{dx}\right|_{\text{normalized}}$$

where h_{avg} is the average grid spacing. The implementation performs this scaling automatically.

Boundary Handling At data boundaries where a full symmetric window cannot be used, the method employs **asymmetric windows**:

Central points (i = offset to n-offset): - Use symmetric window: nl = nr = offset - Pre-computed coefficients applied via fast convolution

Boundary points (left and right edges): - Use asymmetric windows: nl != nr - Coefficients computed per-point for each boundary configuration - Ensures enough points available for polynomial degree - Example: leftmost point uses nl=0, nr=window_size-1

Edge cases: - The window always retains `window_size` points (only its center shifts), and `poly_degree < window_size` is enforced up front, so boundaries always have enough points for the fit - Since an asymmetric window still spans `window_size` points, the boundary pass reuses the coefficient buffers of the central pass instead of allocating per point (v5.11.45) - If coefficient computation fails at a boundary point, the whole call is a hard error (returns `NULL`) rather than silently substituting raw data — matching the central-point path (v5.11.36) - Maintains polynomial exactness property at boundaries

Characteristics Advantages: - Optimal for polynomial signals on uniform grids - Excellent preservation of moments and peak areas - Efficient implementation (convolution) - Minimal phase distortion - Simultaneous computation of functions and derivatives

Disadvantages: - **Requires uniform grid** - automatically rejected if CV > 0.05 - Fixed coefficients for entire window - May introduce oscillations at sharp edges - Limited adaptability - Numerical warnings for degrees > 6

A.3 Tikhonov Regularization (TIKHONOV)

Theoretical Foundation Tikhonov regularization solves the ill-posed inverse smoothing problem using a variational approach. We seek a function minimizing the functional:

Continuous formulation:

$$J[u] = \underbrace{\int (y(x) - u(x))^2 \, dx}_{\text{Data fidelity}} + \lambda \underbrace{\int (u''(x))^2 \, dx}_{\text{Smoothness penalty}}$$

Discrete formulation:

$$J[\mathbf{u}] = \|\mathbf{y} - \mathbf{u}\|^2 + \lambda \|D^2\mathbf{u}\|^2$$

where: - $\|\mathbf{y} - \mathbf{u}\|^2 = \sum (y_i - u_i)^2$ is the **data fidelity term** - $\|D^2\mathbf{u}\|^2 = \sum (D^2 u_i)^2$ is the **regularization term** (smoothness penalty) - λ is the **regularization parameter** controlling the balance - D^2 is the discrete second derivative operator

The Regularization Parameter lambda The parameter lambda controls the balance between fitting the data and smoothing the result.

Physical Interpretation:

λ	Effect	Functional
$\lambda = 0$	No smoothing, $u = y$	$J[u] = \|\mathbf{y} - \mathbf{u}\|^2$ only
$\lambda \to \infty$	Maximum smoothing, $u \to$ straight line	$J[u] \approx \lambda \|D^2\mathbf{u}\|^2$ dominates
λ optimal	Balanced data fit and smoothness	Both terms contribute meaningfully

Mathematical Role:

The minimization of $J[\mathbf{u}]$ leads to:

$$(I + \lambda (D^2)^T W D^2)\, \mathbf{u} = \mathbf{y}$$

Effect of λ on the solution: - **Small λ (< 0.01):** Matrix $\approx I$ → solution $\mathbf{u} \approx \mathbf{y}$ (minimal smoothing) - **Large λ (> 1.0):** Matrix $\approx \lambda (D^2)^T W D^2$ → strong curvature penalty (heavy smoothing) - **Optimal lambda:** Matrix components balanced → noise removed, signal preserved

Frequency Domain Interpretation:

In Fourier space, Tikhonov acts as a low-pass filter:

$$\hat{H}(\omega) = \frac{1}{1 + \lambda \omega^4}$$

where ω is spatial frequency.

Effect: - **Low frequencies (slow variations):** $\hat{H} \approx 1$ → preserved - **High frequencies (noise, rapid variations):** $\hat{H} \approx 1/(\lambda\omega^4)$ → attenuated - **Cutoff frequency:** $\omega_c \sim \lambda^{-1/4}$

Larger λ → lower cutoff → more aggressive low-pass filtering → smoother result. Smaller λ → higher cutoff → less filtering → result closer to data.

Lambda and Grid Spacing:

The effective regularization strength depends on grid spacing:

$$\text{Effective strength} \sim \frac{\lambda}{h_{\text{avg}}^3}$$

Same λ on finer grid → weaker smoothing; same λ on coarser grid → stronger smoothing.

For dimensional consistency, λ has units $[\text{Length}^3]$ (it scales the squared second derivative integrated over the grid; the data term is an unweighted sum).

Second Derivative Penalty: $(\mathbf{D^2})^{\mathbf{s}}\mathbf{WD^2}$ Gram Matrix The regularization term $\|D^2\mathbf{u}\|^2$ is discretized using the **Gram matrix** of the second derivative operator D^2. The operator D^2 is an $(n-2) \times n$ matrix — it has rows only for interior points ($k = 1, \ldots, n-2$). This means natural boundary conditions ($D^2 u = 0$ at endpoints) are **implicit**: there are simply no D^2 rows for boundary points.

The penalty matrix is constructed as a sum of rank-1 contributions:

$$(D^2)^T W D^2 = \sum_{k=1}^{n-2} w_k \cdot \mathbf{d}_k^T \cdot \mathbf{d}_k$$

where $\mathbf{d}_k$ is the k-th row of D^2 (a 3-element stencil at positions $k{-}1, k, k{+}1$) and w_k is the integration weight. Since each $\mathbf{d}_k$ touches 3 consecutive points, the Gram matrix is **pentadiagonal** (bandwidth $kd = 2$).

A single discretization scheme is used for **all** grids (uniform and non-uniform alike) — there is no CV-based method switch.

Each interior row k uses the local spacings $h_l = x_k - x_{k-1}$, $h_r = x_{k+1} - x_k$:

$$\mathbf{d}_k = [a_k,\ b_k,\ c_k] \qquad \text{at positions } (k{-}1,\ k,\ k{+}1)$$

where:

$$a_k = \frac{2}{(h_l + h_r) \cdot h_l}, \qquad b_k = \frac{-2}{h_l \cdot h_r}, \qquad c_k = \frac{2}{(h_l + h_r) \cdot h_r}$$

This is the standard second-derivative formula for non-uniform grids derived from Taylor expansion. The integration weight is $w_k = (h_l{+}h_r)/2$, so the penalty approximates the integral $\lambda \int (u'')^2\, dx$.

For each interior point k, the rank-1 outer product $w_k \cdot \mathbf{d}_k^T \mathbf{d}_k$ is accumulated into the pentadiagonal matrix (upper triangle only): diagonal, 1st superdiagonal, and 2nd superdiagonal. The result is symmetric (guaranteed by the Gram structure), positive definite, and pentadiagonal (bandwidth $kd = 2$).

Uniform-grid reduction. When $h_l = h_r = h$ the stencil collapses to the classical 4th-difference $[1, -4, 6, -4, 1] \cdot \lambda/h^3$. **Example** for $n = 6$ with $c = \lambda/h^3$:

$$A = I + \lambda(D^2)^T W D^2 = \begin{pmatrix} 1+c & -2c & c & 0 & 0 & 0 \\ -2c & 1+5c & -4c & c & 0 & 0 \\ c & -4c & 1+6c & -4c & c & 0 \\ 0 & c & -4c & 1+6c & -4c & c \\ 0 & 0 & c & -4c & 1+5c & -2c \\ 0 & 0 & 0 & c & -2c & 1+c \end{pmatrix}$$

Note: Boundary points (first/last rows) receive less regularization because fewer D^2 stencils overlap them.

Natural Boundary Conditions Natural boundary conditions ($D^2u = 0$ at endpoints) are **implicit** in the Gram matrix construction:

- D^2 is an $(n-2) \times n$ matrix with rows only for interior points $k = 1, \ldots, n-2$
- No boundary rows exist in D^2, so no explicit boundary penalty is applied
- Boundary points ($i = 0$, $i = n-1$) are regularized only through their participation in nearby interior stencils
- This approach avoids the need for explicit boundary formulas and produces cleaner edge behavior

Boundary behavior: - Boundary points receive less regularization pressure than interior points - For very small lambda, boundary points may track the data more closely than interior points - This is generally desirable: edges are less constrained, avoiding artificial boundary effects

Functional Computation The actual value of the minimized functional is computed for diagnostic purposes:

Data term:

$$\|\mathbf{y} - \mathbf{u}\|^2 = \sum_{i=0}^{n-1} (y_i - u_i)^2$$

Regularization term (interior points only):

Consistent with the Gram matrix construction, the regularization term sums only over interior points (natural BCs are implicit — no boundary terms), using the same integral-measure weighting as the matrix:

$$\|D^2\mathbf{u}\|_W^2 = \sum_{i=1}^{n-2} (D^2 u_i)^2 \cdot \frac{h_l + h_r}{2}$$

where $D^2 u_i = \frac{2}{h_l + h_r} \left[\frac{u_{i-1}}{h_l} - u_i \left(\frac{1}{h_l} + \frac{1}{h_r} \right) + \frac{u_{i+1}}{h_r} \right]$

Total functional:

$$J[\mathbf{u}] = \|\mathbf{y} - \mathbf{u}\|^2 + \lambda \|D^2 \mathbf{u}\|^2$$

Variational Approach The minimum of functional $J[\mathbf{u}]$ satisfies the Euler-Lagrange equation:

$$\frac{\partial J}{\partial u_i} = 0 \implies -2(y_i - u_i) + 2\lambda \left((D^2)^T W D^2 \, \mathbf{u} \right)_i = 0$$

which leads to the linear system:

$$(I + \lambda (D^2)^T W D^2) \, \mathbf{u} = \mathbf{y}$$

where $(D^2)^T W D^2$ is the Gram matrix of the second derivative operator D^2.

Matrix Representation The linear system $(I + \lambda (D^2)^T W D^2) \, \mathbf{u} = \mathbf{y}$ has matrix $A = I + \lambda (D^2)^T W D^2$ with structure:

Properties: - Symmetric (Gram matrix structure guarantees this) - Positive definite - Pentadiagonal (banded with bandwidth $kd = 2$)

General pentadiagonal form:

$$A = \begin{pmatrix} d_0 & e_0 & f_0 & 0 & \cdots & 0 \\ e_0 & d_1 & e_1 & f_1 & \cdots & 0 \\ f_0 & e_1 & d_2 & e_2 & \cdots & 0 \\ 0 & f_1 & e_2 & d_3 & \cdots & 0 \\ \vdots & & & & \ddots & \vdots \\ 0 & 0 & 0 & 0 & e_{n-2} & d_{n-1} \end{pmatrix}$$

where d_i = diagonal, e_i = 1st off-diagonal, f_i = 2nd off-diagonal.

The pentadiagonal structure arises because each row of D^2 touches 3 consecutive points ($k-1, k, k+1$), so the Gram matrix $(D^2)^T D^2$ couples points up to 2 positions apart.

This structure allows efficient solution using LAPACK's banded solver `dpbsv`.

Generalized Cross Validation (GCV) For automatic λ selection (`-l auto`), we minimize the GCV criterion:

$$\text{GCV}(\lambda) = \frac{n \cdot \text{RSS}(\lambda)}{(n - \text{tr}(H_\lambda))^2}$$

where: - $\text{RSS}(\lambda) = \|\mathbf{y} - \mathbf{u}_\lambda\|^2$ is the residual sum of squares - $H_\lambda = (I + \lambda (D^2)^T W D^2)^{-1}$ is the influence matrix (smoother matrix) - $\text{tr}(H_\lambda)$ is the trace (effective number of parameters)

Interpretation: - $\text{tr}(H_\lambda)$ measures model complexity (degrees of freedom) - Small λ: $\text{tr}(H) \approx n$ (interpolation, overfitting) - Large λ: $\text{tr}(H) \to 2$ (linear fit, underfitting — D^2 null space is constants + linear) - Optimal λ: minimizes prediction error

Trace estimation using eigenvalues:

For uniform grids, the eigenvalues of $(D^2)^T D^2$ are the squares of the eigenvalues of $D^T D$ (first-derivative operator):

$$\text{tr}(H_\lambda) \approx 2 + \sum_{k=1}^{n-2} \frac{1}{1 + \lambda \mu_k}$$

where the eigenvalues of $(D^2)^T D^2$ are:

$$\theta_k = \frac{\pi k}{n}, \qquad \mu_k = \left(\frac{4 \sin^2(\theta_k/2)}{h^2} \right)^2$$

The null space of D^2 is 2-dimensional (constants and linear functions), so trace starts at 2.0 (these two modes are unpenalized: $1/(1+0) = 1$ each).

Note: This approximation is exact for uniform grids but approximate for non-uniform grids. For highly non-uniform grids (CV > 0.2), the program issues a warning.

Size-dependent refinement:

One log-spaced GCV scan serves every dataset size; only the refinement step depends on n:

Size	Trace estimator	Lambda search
$n \leq 5000$	Eigenvalue sum above	13-point log scan over $[10^{-8}, 10^{0}]$ + 8-point refinement around the minimum
$n > 5000$	Eigenvalue sum above	13-point log scan only (no refinement)

The eigenvalue sum is $O(n)$ per λ candidate — the same order as the band solve itself — so it is used for all dataset sizes. The refinement step at $n \leq 5000$ adds 8 extra λ evaluations clustered around the coarse-scan minimum (factors $0.3 \ldots 1.7$). Datasets that straddle the threshold may therefore receive slightly different optimal λ from otherwise-identical inputs.

Because λ is dimensional (it scales with h^3 and the squared amplitude of y), the fixed search range $[10^{-8}, 10^{0}]$ may not match the scale of every dataset. If the selected optimum lies at the edge of the range, a warning is printed and λ should be set manually with `-l`.

Characteristics Advantages: - Global optimization with theoretical foundation - Flexible balance between data fidelity and smoothness (controlled by lambda) - Robust to outliers (quadratic penalty less sensitive than least squares) - Efficient for large datasets (O(n) memory and time) - **Automatic lambda selection via GCV** - no guessing needed - **Excellent for non-uniform grids** - correct discretization automatic - **Unified approach** - same algorithm for uniform and non-uniform grids - Works well for noisy data with global trends

Disadvantages: - Single global parameter lambda (cannot vary locally) - May suppress local details if lambda too large - GCV may fail for some data types (especially highly non-uniform grids) - Requires LAPACK library - Boundary effects if data has discontinuities at edges

---

A.4 Butterworth Filter (BUTTERWORTH)

Theoretical Foundation The Butterworth filter is a classical **low-pass frequency filter** in digital signal processing (DSP). It removes high-frequency noise while preserving low-frequency signal trends.

The filter is characterized by a **maximally flat magnitude response** in the passband and provides zero phase distortion when implemented as filtfilt.

Filter Transfer Function:

In the analog domain (s-domain), the Butterworth filter has magnitude response:

$$|H(j\omega)|^2 = \frac{1}{1 + (\omega/\omega_c)^{2N}}$$

where: - N = filter order (4 in our implementation) - ω_c = cutoff frequency (3dB point) - ω = frequency

Key Properties: - **Maximally flat passband:** No ripples for omega < omega_c - **Monotonic rolloff:** Smooth transition from passband to stopband - **-3dB at cutoff:** |H(j omega_c)| = 1/sqrt(2) ~ 0.707 - **Rolloff rate:** -20N dB/decade (for N=4: -80 dB/decade)

Digital Implementation The smooth program implements a **4th-order digital Butterworth low-pass filter** using the following algorithm:

Step 1: Pole Calculation

Butterworth poles lie on unit circle in s-domain at angles:

$$\theta_k = \frac{\pi}{2} + \frac{\pi(2k+1)}{2N}, \qquad k = 0, 1, \ldots, N-1$$

For $N = 4$:

$$s_{\text{poles}}[k] = e^{j\theta_k} \qquad \text{where } \theta \in \left\{ \frac{5\pi}{8}, \frac{7\pi}{8}, \frac{9\pi}{8}, \frac{11\pi}{8} \right\}$$

Step 2: Frequency Scaling

Scale poles by prewarped cutoff frequency:

$$\omega_c = \tan(\pi f_c/2), \qquad s_{\text{scaled}} = \omega_c \cdot s_{\text{poles}}$$

Prewarping correction: The bilinear transform introduces frequency warping. The factor $\tan(\pi f_c/2)$ compensates for this, ensuring the digital filter's cutoff matches the desired normalized frequency f_c.

Step 3: Bilinear Transform

Convert analog poles to digital domain:

$$z_{\text{poles}} = \frac{2 + s_{\text{scaled}}}{2 - s_{\text{scaled}}}$$

The bilinear transformation maps: - Left half of s-plane → inside unit circle in z-plane - j-omega axis → unit circle in z-plane - Preserves stability

Step 4: Biquad Cascade

Form two 2nd-order sections (biquads) from conjugate pole pairs:

$$H(z) = H_1(z) \cdot H_2(z), \qquad H_i(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}$$

This approach provides better numerical stability than direct 4th-order implementation.

Filtfilt Algorithm The **filtfilt** (forward-backward filtering) eliminates phase distortion:

Algorithm: 1. **Pad signal:** Reflect signal at boundaries; the padding length is sized to absorb the filter transient ($\sim 5/(1 - r_{\text{max}})$, where r_{max} is the slowest pole radius, floored at 14 and capped at $n - 1$), so it grows automatically as f_c shrinks 2. **Forward filter:** Apply H(z) from left to right → y_fwd 3. **Reverse:** y_rev = reverse(y_fwd) 4. **Backward filter:** Apply H(z) to y_rev → y_bwd 5. **Reverse back:** y_final = reverse(y_bwd) 6. **Extract:** Remove padding to get final result

Effect: - **Zero phase lag:** No signal delay - **Effective order:** $2N = 8$ (squared magnitude response) - **Steeper rolloff:** $|H_{\text{eff}}(j\omega)|^2 = |H(j\omega)|^4$

Initial Conditions (Biquad IC) To minimize edge transients, we compute initial filter state for each **biquad section** using an **analytical solution**:

Problem: For each 2nd-order biquad section, find initial state $\mathbf{z}_i$ such that for constant input $x = c$:

$$\mathbf{z}_i = A \cdot \mathbf{z}_i + B \cdot c$$

This ensures each biquad starts in steady-state, eliminating startup transients.

Solution for 2nd-order biquad: Solve the 2×2 linear system analytically:

$$(I - A) \cdot \mathbf{z}_i = \mathbf{B}$$

where for Transposed Direct Form II:

$$I - A = \begin{pmatrix} 1 + a_1 & -1 \\ a_2 & 1 \end{pmatrix}, \qquad \mathbf{B} = \begin{pmatrix} b_1 - a_1 b_0 \\ b_2 - a_2 b_0 \end{pmatrix}$$

Analytical solution using Cramer's rule:

$$\det(I - A) = 1 + a_1 + a_2$$

$$z_i[0] = \frac{B_0 + B_1}{\det}, \qquad z_i[1] = \frac{-a_2 B_0 + (1 + a_1) B_1}{\det}$$

Implementation advantages: - **No LAPACK dependency** for initial conditions (purely analytical) - **Per-biquad computation** - simple 2x2 systems instead of 4x4 - **Numerically robust** - direct formulas avoid iterative solvers - **Efficient** - closed-form solution, no matrix decomposition needed

Biquad Cascade Implementation The 4th-order Butterworth filter is implemented as a **cascade of 2 second-order sections (biquads)** for superior numerical stability.

Why biquad cascade? - **Numerical stability:** Each 2nd-order section has well-conditioned coefficient magnitudes - **Reduced quantization errors:** Coefficients stay in reasonable ranges (~0.1 to 10) - **Industry standard:** Used in professional DSP applications - **Modular:** Each biquad can be analyzed and tested independently

Each biquad section uses **Transposed Direct Form II** (TDF-II) for optimal numerical properties:

$$\begin{aligned} y[n] &= b_0 \cdot x[n] + z_0 \\ z_0 &= b_1 \cdot x[n] - a_1 \cdot y[n] + z_1 \\ z_1 &= b_2 \cdot x[n] - a_2 \cdot y[n] \end{aligned}$$

where **z** is the biquad state (2 elements per section), $[b_0, b_1, b_2]$ are numerator coefficients, and $[1, a_1, a_2]$ are denominator coefficients (a_0 normalized to 1).

Normalized Cutoff Frequency Technical details:

$$f_c = \frac{f_{\text{cutoff}}}{f_{\text{Nyquist}}} = \frac{f_{\text{cutoff}}}{f_{\text{sample}}/2}, \qquad f_{\text{sample}} = \frac{1}{h_{\text{avg}}}$$

Nyquist Constraint: $0 < f_c < 1.0$ - $f_c = 1.0$ corresponds to Nyquist frequency ($f_{\text{sample}}/2$) - maximum possible - Higher fc → less filtering (more high frequencies pass) - Lower fc → more filtering (smoother result)

Grid Requirements Butterworth filter works best with **uniform or nearly-uniform grids**. The filter assumes uniform sampling when computing the cutoff frequency. For non-uniform grids, use Tikhonov method (`-m 2 -l auto`) which handles arbitrary spacing correctly.

Characteristics Advantages: - **Zero phase distortion** (filtfilt eliminates all phase lag) - **Maximally flat frequency response** in passband - **Superior numerical stability** (biquad cascade implementation) - **No LAPACK dependency** for initial conditions (analytical solution) - **Classical DSP approach** with extensive literature and understanding - **Predictable frequency-domain behavior** - easy to interpret cutoff frequency - **No ringing** (unlike Chebyshev or elliptic filters) - **Efficient implementation** - O(n) time complexity - **Smooth monotonic rolloff** - natural attenuation curve - **Robust for extreme cutoffs** (fc < 0.05 handled well by biquad cascade)

Disadvantages: - **Requires uniform/nearly-uniform grid** (CV < 0.15 enforced, warning for CV > 0.05) - **Less local adaptability** than polynomial methods - **Frequency-domain assumption** — presumes near-uniform sampling - **Edge effects** despite padding - **Frequency interpretation** may be less intuitive than lambda for some users

---

Appendix B: Implementation Details

Data Structures

```c
// Polynomial fitting result
typedef struct {
    double *y_smooth;     // Smoothed values
    double *y_deriv;      // First derivatives
    int n;                // Number of points
    int poly_degree;      // Polynomial degree
    int window_size;      // Window size
} PolyfitResult;
```

```c
// Savitzky-Golay result
typedef struct {
    double *y_smooth;     // Smoothed values
    double *y_deriv;      // First derivatives
    int n;                // Number of points
    int poly_degree;      // Polynomial degree
    int window_size;      // Window size
} SavgolResult;

// Savitzky-Golay coefficient computation (internal/static; returns 0 on
// success, -1 on error, with the output array zeroed for safety)
static int savgol_coefficients(int nl, int nr, int poly_degree,
                               int deriv_order, double *c);

// Tikhonov result
typedef struct {
    double *y_smooth;           // Smoothed values
    double *y_deriv;            // First derivatives
    double lambda;              // Used parameter
    int n;                      // Number of points
    double data_term;           // ||y - u||^2
    double regularization_term; // lambda||D^2 u||^2
    double total_functional;    // J[u]
} TikhonovResult;

// Tikhonov functions
TikhonovResult* tikhonov_smooth(const double *x, const double *y, int n, double lambda,
                                const GridAnalysis *grid_info);
double find_optimal_lambda_gcv(const double *x, const double *y, int n,
                               const GridAnalysis *grid_info);
void free_tikhonov_result(TikhonovResult *result);

// Butterworth biquad section
typedef struct {
    double b[3];  // Numerator coefficients: [b0, b1, b2]
    double a[3];  // Denominator coefficients: [a0=1, a1, a2]
} BiquadSection;

// Butterworth result
typedef struct {
    double *y_smooth;     // Smoothed values
    double *y_deriv;      // First derivatives (5-point stencils)
    int n;                // Number of points
    int order;            // Filter order (BUTTERWORTH_ORDER = 4)
    double cutoff_freq;   // Normalized cutoff frequency (0 < fc < 1)
    double sample_rate;   // Effective sample rate from data spacing
} ButterworthResult;
```

```c
// Butterworth functions
ButterworthResult* butterworth_filtfilt(const double *x, const double *y, int n,
                                        double cutoff_freq, int auto_cutoff,
                                        const GridAnalysis *grid_info);
// Automatic cutoff (Morozov); internal/static, operates on normalized freqs
static double estimate_cutoff_frequency(const double *y, int n);
void free_butterworth_result(ButterworthResult *result);

// Grid analysis
typedef struct {
    double h_min;           // Minimum spacing
    double h_max;           // Maximum spacing
    double h_avg;           // Average spacing
    double h_std;           // Standard deviation
    double ratio_max_min;   // h_max/h_min ratio
    double cv;              // Coefficient of variation
    double uniformity_score;// Uniformity score (0-1)
    int is_uniform;         // 1 = uniform, 0 = non-uniform
    int n_clusters;         // Number of detected clusters
    int reliability_warning;// Reliability warning
    char warning_msg[512];  // Warning text
    int n_points;           // Number of points
} GridAnalysis;

// Grid analysis functions
GridAnalysis* analyze_grid(const double *x, int n);
const char* get_grid_recommendation(GridAnalysis *analysis);
void print_grid_analysis(GridAnalysis *analysis, int verbose, const char *prefix);
void free_grid_analysis(GridAnalysis *analysis);
```

LAPACK Routines Used

Method	Routine	Purpose
POLYFIT	`dgelss`	SVD least squares solver (Vandermonde system)
SAVGOL	`dposv`	Symmetric positive definite solver (coefficient computation)
TIKHONOV	`dpbsv`	Banded symmetric positive definite solver (pentadiagonal, kd=2)
BUTTERWORTH	None	Analytical biquad IC via Cramer's rule

LAPACK banded storage for Tikhonov (`dpbsv`):

```
// Banded matrix storage (LAPACK column-major format)
// For pentadiagonal symmetric matrix A with bandwidth kd=2:
//
//     [ AB[0,j] ]  = 2nd superdiagonal elements  (a[i,i+2])
//     [ AB[1,j] ]  = 1st superdiagonal elements  (a[i,i+1])
//     [ AB[2,j] ]  = diagonal elements            (a[i,i])
//
// Storage layout for pentadiagonal matrix:
//
//         [ *    *    a02  a13  a24  ... ]   <- row 0 (2nd superdiagonal)
//         [ *    a01  a12  a23  a34  ... ]   <- row 1 (1st superdiagonal)
//    AB = [ a00  a11  a22  a33  a44  ... ]   <- row 2 (diagonal)
//
// System solution (kd=2, ldab=3)
dpbsv_(&uplo, &n, &kd, &nrhs, AB, &ldab, b, &n, &info);
```

POLYFIT SVD solver:

```
// Build Vandermonde matrix
build_vandermonde(x, i - offset, i + offset, x[i], poly_degree, V, window_size);

// Solve least squares using SVD decomposition
dgelss_(&window_size, &matrix_cols, &nrhs, V, &window_size,
        rhs, &rhs_size, sing_vals, &rcond, &effective_rank,
        work, &lwork, &info);

// Extract solution: rhs[0] = a_0 (value), rhs[1] = a_1 (derivative)
result->y_smooth[i] = rhs[0];
result->y_deriv[i] = (poly_degree > 0) ? rhs[1] : 0.0;
```

Numerical diagnostics (POLYFIT): - Tracks the effective condition number $\kappa = \sigma_{\max}/\sigma_{\min}$ (after `rcond` truncation) across every interior window and reports the worst one at the end of the run - The report is only issued if $\kappa > 10^8$, as a warning about potential numerical issues - Reports how many windows came out rank-deficient - Falls back to the original value with zero derivative if SVD fails, both at interior windows and at the boundary points that would have been extrapolated from them

SAVGOL coefficient solver:

```
// Solve linear system for Savitzky-Golay coefficients
dposv_(&uplo, &matrix_size, &nrhs, A, &matrix_size, B, &matrix_size, &info);
```

File Structure

```
smooth/
|--- smooth.c           # Main program
|--- polyfit.c/h        # Polynomial fitting module
|--- savgol.c/h         # Savitzky-Golay module
|--- tikhonov.c/h       # Tikhonov regularization module
|--- butterworth.c/h    # Butterworth filter module
```

```
|--- grid_analysis.c/h  # Grid analysis module
|--- timestamp.c/h      # Timestamp parsing module
|--- parser.c/h      # Input parser (tokenizing, `#` comments, column/timestamp model)
|--- revision.h         # Program version
|--- Makefile           # Build system with test targets
|--- README.md          # This documentation
+--- tests/             # Unit testing framework (Unity)
    |--- unity.c/h               # Unity testing framework
    |--- unity_internals.h       # Unity internals
    |--- test_main.c             # Test runner (116 tests)
    |--- test_grid_analysis.c    # Grid analysis tests (7 tests)
    |--- test_polyfit.c          # Polyfit module tests (21 tests)
    |--- test_savgol.c           # Savgol module tests (16 tests)
    |--- test_tikhonov.c         # Tikhonov module tests (25 tests)
    |--- test_butterworth.c      # Butterworth module tests (22 tests)
    |--- test_timestamp.c        # Timestamp module tests (18 tests)
    +--- test_parser.c           # Input parser tests (7 tests, end-to-end)
```

Version History

The full, per-release version history lives in `revision.h` — the comment block at the top of the file, newest first. It is the single source of truth; it was duplicated here and the copy had already drifted out of date.

To read it:

```
head -n 60 revision.h        # most recent releases
./smooth -h | head -n 6      # version and date of the built binary
```

Document revision: 2026-07-26 **Program version:** see `revision.h` (or `./smooth -h`) **Dependencies:** LAPACK, BLAS **Testing framework:** Unity (included in tests/) **License:** MIT License